



ArchivRH

Guide d'installation locale & de déploiement — Port Autonome de Douala / DRH

Ce document explique comment **(A)** récupérer le projet depuis Git et le faire fonctionner sur votre ordinateur en local, puis **(B)** le déployer sur le serveur interne de l'entreprise — au choix sous **Linux (Ubuntu)** ou **Windows Server**. Les commandes sont à copier-coller dans un terminal.

Pré-requis communs : Git, Python 3.11+, Node.js 20+, PostgreSQL 15. Dépôt : <https://github.com/alexD237/test.git> — branche `claude/project-file-storage-2k5vmb`.

Section A — Faire tourner le projet en local

1. Récupérer le code

```
# Cloner le dépôt et se placer sur la bonne branche
git clone https://github.com/alexD237/test.git
cd test
git checkout claude/project-file-storage-2k5vmb
cd gestcourrier
```

2. Créer la base de données PostgreSQL

```
sudo -u postgres psql -c "CREATE DATABASE gestcourrier;"
sudo -u postgres psql -c "CREATE USER gestcourrier WITH PASSWORD 'choisir-un-mdp';"
sudo -u postgres psql -c "GRANT ALL PRIVILEGES ON DATABASE gestcourrier TO gestcourrier;"
sudo -u postgres psql -d gestcourrier -c "GRANT ALL ON SCHEMA public TO gestcourrier;"
```

(Sous Windows, utilisez « SQL Shell (psql) » ou pgAdmin et tapez les mêmes ordres SQL.)

3. Lancer le backend (Django)

```
cd backend
python3 -m venv venv
source venv/bin/activate           # Windows : venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env              # puis éditez .env (voir tableau)
python manage.py migrate
python manage.py createsuperuser   # crée votre 1er compte administrateur
python manage.py runserver         # API sur http://localhost:8000
```

Dans `.env`, pour le local renseignez au minimum :

Variable	Valeur en local
SECRET_KEY	une longue chaîne aléatoire

DEBUG	True
DB_NAME / DB_USER	gestcourrier
DB_PASSWORD	le mot de passe choisi à l'étape 2

Générer une clé secrète : `python -c "import secrets; print(secrets.token_urlsafe(50))"`

4. Lancer le frontend (React) — dans un 2e terminal

```
cd gestcourrier/frontend
npm install
npm run dev                # interface sur http://localhost:5173
```

Ouvrez `http://localhost:5173` et connectez-vous avec le compte créé à l'étape 3. Vite redirige automatiquement les appels `/api` vers le backend (port 8000).

Export PDF (reporting). La génération des rapports utilise WeasyPrint, qui requiert Pango/Cairo. Ubuntu/Debian : `sudo apt install libpango-1.0-0 libpangocairo-1.0-0 libgdk-pixbuf-2.0-0` ; macOS : `brew install pango` ; Windows : installez le runtime GTK3 (voir Section B.2).

Section B — Déployer sur le serveur de l'entreprise

Architecture identique sur les deux OS : **Nginx** sert l'interface React et proxifie l'API **Django** ; **PostgreSQL** stocke les données ; les PDF restent sur le serveur et ne sont jamais exposés directement. Choisissez la variante correspondant à votre serveur. Les fichiers de configuration sont fournis dans `gestcourrier/deploy/`.

B.1 — Serveur Linux (Ubuntu 22.04)

1. Préparer le serveur

```
sudo apt update
sudo apt install -y git python3-venv python3-pip nodejs npm postgresql nginx \
    libpango-1.0-0 libpangocairo-1.0-0 libgdk-pixbuf-2.0-0
```

2. Récupérer le code dans `/opt/gestcourrier`

```
sudo git clone https://github.com/alexD237/test.git /opt/archivrh-src
sudo mv /opt/archivrh-src/gestcourrier /opt/gestcourrier
sudo useradd --system gestcourrier    # compte de service dédié
```

3. Base de données

Mêmes commandes qu'en Section A, étape 2.

4. Backend + service Gunicorn

```
cd /opt/gestcourrier/backend
python3 -m venv venv && source venv/bin/activate
pip install -r requirements.txt
cp .env.example .env                # DEBUG=False, ALLOWED_HOSTS, SECRET_KEY, DB_*
```

```
python manage.py migrate
python manage.py collectstatic --noinput
python manage.py createsuperuser

sudo cp /opt/gestcourrier/deploy/gunicorn.service /etc/systemd/system/gestcourrier.service
sudo mkdir -p /var/log/gestcourrier
sudo chown -R gestcourrier /opt/gestcourrier /var/log/gestcourrier
sudo systemctl enable --now gestcourrier
```

5. Frontend (build servi par Nginx)

```
cd /opt/gestcourrier/frontend
npm ci && npm run build # génère frontend/dist
```

6. Nginx

```
sudo cp /opt/gestcourrier/deploy/nginx.conf /etc/nginx/sites-available/gestcourrier
sudo ln -s /etc/nginx/sites-available/gestcourrier /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx
```

B.2 — Serveur Windows Server

Différences clés : Gunicorn ne fonctionne pas sous Windows — on utilise **Waitress** (serveur WSGI pur Python) lancé en service via **NSSM** ; Nginx existe en build Windows.

1. Pré-requis (installeurs Windows)

- Python 3.11 (cochez « Add Python to PATH »), Node.js LTS, PostgreSQL 15.
- Nginx pour Windows (nginx.org), NSSM (gestion de services, nssm.cc).
- Runtime **GTK3** (pour WeasyPrint / export PDF du reporting).

2. Récupérer le code (PowerShell)

```
git clone https://github.com/alexD237/test.git C:\archivrh
cd C:\archivrh
git checkout claudes/project-file-storage-2k5vmb
```

3. Base de données

Via « SQL Shell (psql) » ou pgAdmin, exécutez les mêmes ordres SQL qu'en Section A étape 2 (`CREATE DATABASE` , `CREATE USER` , `GRANT`).

4. Backend (Waitress)

```
cd C:\archivrh\gestcourrier\backend
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
pip install waitress # serveur WSGI compatible Windows
copy .env.example .env # DEBUG=False, ALLOWED_HOSTS, SECRET_KEY, DB_*
python manage.py migrate
python manage.py collectstatic --noinput
python manage.py createsuperuser
```

```
# Test manuel :
venv\Scripts\waitress-serve --listen=127.0.0.1:8000 config.wsgi:application
```

5. Installer l'API en service Windows (NSSM)

```
nssm install ArchivRH "C:\archivrh\gestcourrier\backend\venv\Scripts\waitress-serve.exe" ^
"--listen=127.0.0.1:8000 config.wsgi:application"
nssm set ArchivRH AppDirectory C:\archivrh\gestcourrier\backend
nssm start ArchivRH
```

6. Frontend

```
cd C:\archivrh\gestcourrier\frontend
npm ci
npm run build
```

7. Nginx pour Windows

Dans `C:\nginx\conf\nginx.conf`, placez ce bloc `server` (chemins en slash avant, acceptés par Nginx sous Windows) :

```
server {
    listen 80;
    server_name gestcourrier.drh.pad.local;
    client_max_body_size 25M;

    location /api/ { proxy_pass http://127.0.0.1:8000;
                    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
                    proxy_set_header X-Forwarded-Proto $scheme; }
    location /admin/ { proxy_pass http://127.0.0.1:8000; }
    location /static/ { alias C:/archivrh/gestcourrier/backend/staticfiles/; }

    root C:/archivrh/gestcourrier/frontend/dist;
    index index.html;
    location / { try_files $uri $uri/ /index.html; }
}
```

```
# Lancer Nginx, et l'installer aussi en service :
nssm install ArchivRH-Nginx "C:\nginx\nginx.exe"
nssm set ArchivRH-Nginx AppDirectory C:\nginx
nssm start ArchivRH-Nginx
```

HTTPS & WeasyPrint (Windows). Pour le HTTPS, ajoutez un bloc `listen 443 ssl;` avec le certificat de l'entreprise. Tant que TLS n'est pas en place, servez en interne sans activer la redirection HTTPS. Si l'export PDF échoue, c'est que le runtime GTK3 manque : installez-le, le reste de l'application fonctionne sans lui.

Exploitation

Variables d'environnement en production (`.env`, sur les deux OS) : `DEBUG=False` et `ALLOWED_HOSTS=gestcourrier.drh.pad.local` (nom DNS interne). Avec `DEBUG=False`, la redirection HTTPS, HSTS et les cookies sécurisés s'activent automatiquement.

Mise à jour ultérieure (Linux ; sous Windows, remplacez les chemins et `systemctl restart` par `nssm restart ArchivRH`) :

```
cd /opt/gestcourrier && sudo git pull
source backend/venv/bin/activate
pip install -r backend/requirements.txt
python backend/manage.py migrate
python backend/manage.py collectstatic --noinput
( cd frontend && npm ci && npm run build )
sudo systemctl restart gestcourrier && sudo systemctl reload nginx
```

Rappels sécurité (déjà en place). PDF accessibles uniquement via l'API authentifiée · verrouillage de compte après 5 échecs · déconnexion auto après 30 min · journal d'audit immuable · dépendances et polices embarquées (aucun accès Internet requis au runtime).

Pour l'utilisation fonctionnelle (dépôt, recherche, reporting, administration), voir le **Guide utilisateur** : `gestcourrier/docs/GUIDE_UTILISATEUR.md`.